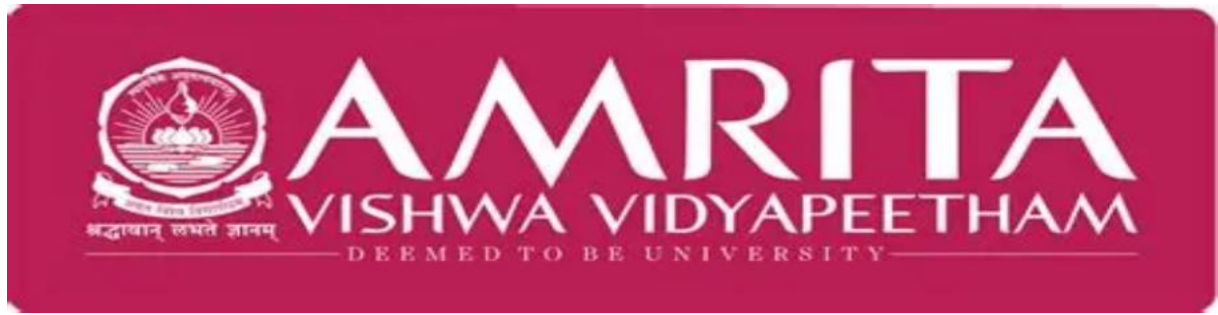




DATASTRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

Submitted by	
Name	NADIPALLI HASINI
Roll No	AV.SC.U4CSE25133
Year/Sem/Section	2nd year/Sem-3/CSE-B
Credits	05

Submitted to	
Name	Dr. RAJ KUMAR BATCHU
Department	School of Computing
Designation	Asst.Professor(SL Grade)

INDEX

Sno	Program	Page No.	Remarks
1.	WEEK 1: A. A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”. B. The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n (power), where p=principal growth factor, n= number of years. C. An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. D. In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula($n!$). develop a recursive program to calculate the factorial of a given number. E. Write a recursive python program to print the first n terms of the Fibonacci series.		
2.	WEEK 2: A. Write a program to perform linear search on a given input. B. (i) Write a python program to implement binary search with sorted array as an input. (ii) Write a python program to implement binary search without sorted array as an input		
3.	WEEK 3:		

	A. Write a python program to implement bubble sort, where the input elements are taken from the user. B. Write a python program to implement insertion sort, where the input elements are taken from the user. C. Write a python program to implement selection sort, where the input elements are taken from the user.		
--	--	--	--

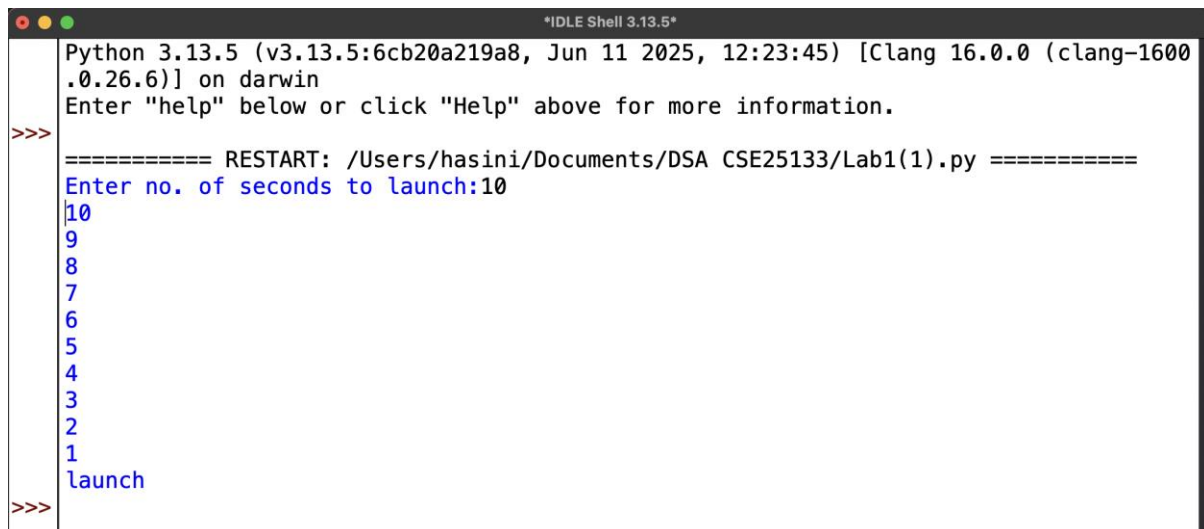
LAB-1

A.AIM: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):  
    for i in range (n, -1,-1):  
        if i==0:  
            print("launch")  
        else:  
            print(i)  
  
n=int (input ("Enter no. of seconds to launch:"))  
launch(n)
```

OUTPUT:



```
*IDLE Shell 3.13.5*  
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600  
.0.26.6)] on darwin  
Enter "help" below or click "Help" above for more information.  
>>>  
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(1).py =====  
Enter no. of seconds to launch:10  
10  
9  
8  
7  
6  
5  
4  
3  
2  
1  
launch  
>>>
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error	Use 4 spaces for each level of indentation.
2.	Type Error	Convert the input to an integer using <code>n = int (input ("Enter no. of seconds to launch: "))</code> .

B.AIM: The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n (power), where p =principal growth factor, n = number of years.

PROGRAM:

```
p = float(input("Enter principal growth factor (p): "))
```

```
n = int(input("Enter number of years (n): "))
```

```
def power (p, n):
```

```
    if n == 0:
```

```
        return 1
```

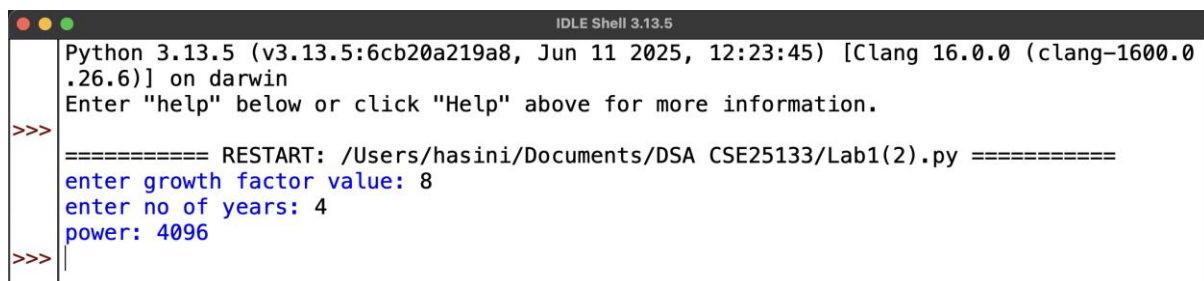
```
    else:
```

```
        return p * power (p, n - 1)
```

```
result = power (p, n)
```

```
print ("Power: ", result)
```

OUTPUT:



```
IDLE Shell 3.13.5
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(2).py =====
enter growth factor value: 8
enter no of years: 4
power: 4096
>>>
```

EXPLANATION:

- The function of power (p , n) calculates p^n using recursion by multiplying p with the result of power (p , $n-1$).
- The recursion stops when n becomes 0, returning 1, which is the base case.
- The final result is displayed after all recursive calls are completed.

ERROR TABLE:

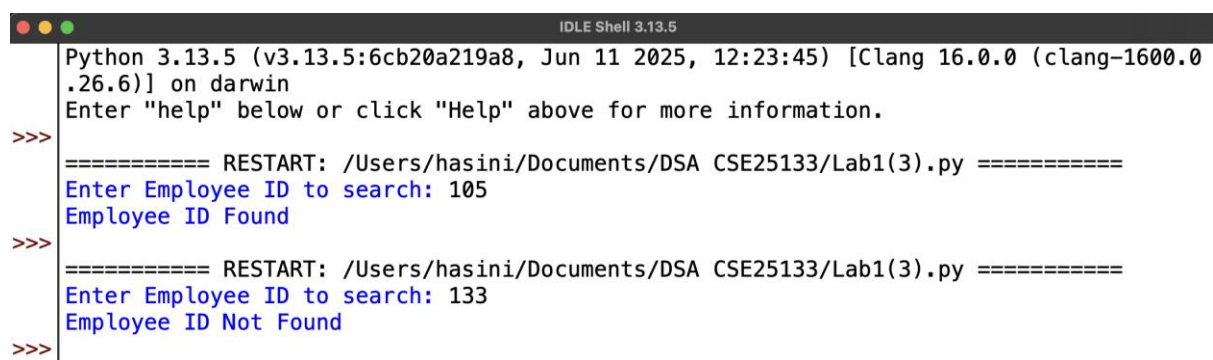
S.no.	Errors	Rectification
1.	Type Error	Convert the input to float or int using float (input()) or int (input ()).
2.	Incorrect output if the recursive call is written as power (p, n + 1) instead of power (p, n - 1).	Change the recursive call to power (p, n - 1) so that n decreases toward the base case.

C.AIM: An HIR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search (list, key, index):  
    if index >= len(list):  
        return False  
    if list[index] == key:  
        return True  
    return search (lst, key, index + 1)  
  
employees = [101, 102, 103, 104, 105]  
key = int (input ("Enter Employee ID to search: "))  
if search (employees, key, 0):  
    print ("Employee ID Found")  
else:  
    print ("Employee ID Not Found")
```

OUTPUT:



```
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin  
Enter "help" below or click "Help" above for more information.  
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(3).py =====  
Enter Employee ID to search: 105  
Employee ID Found  
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(3).py =====  
Enter Employee ID to search: 133  
Employee ID Not Found  
>>>
```

EXPLANATION:

- The recursive function search () checks each employee ID in the list one by one, starting from the given index.
- If the current employee ID matches the required ID (key), it returns True.

- If the ID does not match, the function calls itself recursively with the next index (index + 1).
- If the index reaches the end of the list, it returns False, indicating that the employee ID was not found.
- Finally, the if-else statement displays whether the employee ID is **Found** or **Not Found**.

ERROR TABLE:

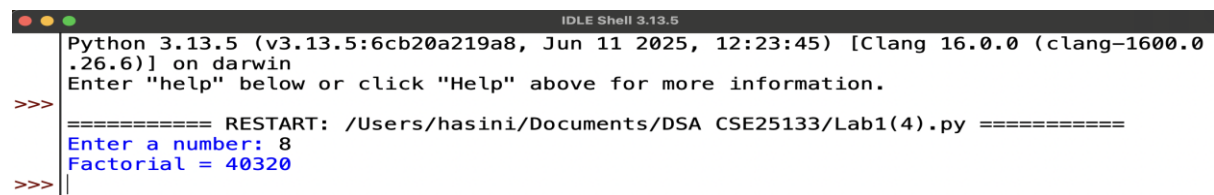
S.no.	Errors	Rectification
1.	Name Error if the list name is misspelled (e.g., lst instead of list).	Use the correct variable name consistently throughout the program.
2.	Syntax Error due to a missing colon (:) after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

D.AIM: In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula($n!$). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial (n - 1)  
  
num = int (input ("Enter a number: "))  
  
if num < 0:  
    print ("Factorial does not exist for negative numbers.")  
else:  
    print ("Factorial =", factorial(num))
```

OUTPUT:



```
IDLE Shell 3.13.5  
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0  
.26.6)] on darwin  
Enter "help" below or click "Help" above for more information.  
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(4).py =====  
Enter a number: 8  
Factorial = 40320  
>>> |
```

EXPLANATION:

- The function `factorial(n)` computes the factorial by multiplying the current number with the factorial of the previous number.
- The recursion ends when n becomes 0 or 1, where the factorial value is 1.
- The program also checks for negative inputs since factorials are defined only for non-negative integers.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Name Error occurs when a variable or function is used without defining it first.	Define the variable or function before using it in the program.
2.	Value Error occurs when an invalid value, such as a letter, is entered where a number is expected.	Enter a valid numerical value or use proper input validation.

E.AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
n = int(input("enter number of terms: "))

def fibonacci(num):

    if num==0:

        return 0

    elif num==1:

        return 1

    else:

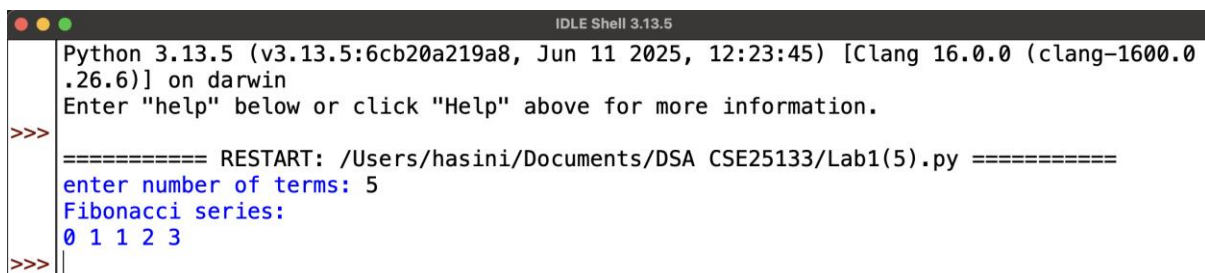
        return fibonacci(num-1) + fibonacci(num-2)

print("Fibonacci series: ")

for i in range(n):

    print(fibonacci(i), end=" ")
```

OUTPUT:

A screenshot of a Python IDLE Shell window. The title bar reads "IDLE Shell 3.13.5". The window contains the following text: "Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin", "Enter 'help' below or click 'Help' above for more information.", a prompt ">>>" followed by "==== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(5).py =====", the input "enter number of terms: 5", the output "Fibonacci series:", and the output "0 1 1 2 3". The prompt ">>>" is visible at the bottom left of the shell area.

```
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>==== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab1(5).py =====
enter number of terms: 5
Fibonacci series:
0 1 1 2 3
>>>
```

EXPLANATION:

- The function `Fibonacci(n)` returns the n th Fibonacci number by recursively adding the previous two Fibonacci numbers.
- The base cases return `0` for the first term and `1` for the second term.
- A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

LAB-2

A.AIM: Write a program to perform linear search on a given input.

PROGRAM:

```
def linear_search(arr,key):  
    for i in range(len(arr)):  
        if arr[i]==key:  
            return i  
    return -1  
  
n = int (input ("Enter no. of elements:"))  
arr= []  
print ("Enter the elements:")  
for i in range(n):  
    arr.append(int(input ()))  
key=int (input ("Enter element to search:"))  
result=linear_search(arr,key)  
if result! =-1:  
    print ("Element found at index", result)  
else:  
    print ("Element not found.")
```

OUTPUT:

```
Python 3.13.5 (v3.13.5:6cb20a219a8, Jun 11 2025, 12:23:45) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(a).py =====
Enter no. of elements:5
Enter the elements:
10
3
8
15
20
Enter element to search:15
Element found at index 3
>>>
```

EXPLANATION:

- The `linear_search()` function checks each element of the list one by one and compares it with the given key.
- If the key is found, its index is returned; otherwise, the function returns - 1.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when <code>int (input ())</code> is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if <code>len(arr)</code> is used on a variable that is not a list.	Make sure <code>arr</code> is initialized as a list using <code>arr = []</code> and elements are appended to it.

B(i). AIM: Write a python program to implement binary search with sorted array as an input.

PROGRAM:

```
def binary_search(arr, key):  
    low = 0  
    high = len(arr) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == key:  
            return mid  
        elif arr[mid] < key:  
            low = mid + 1  
        else:  
            high = mid - 1  
    return -1  
  
n = int (input ("Enter number of elements: "))  
arr = []  
print ("Enter elements in sorted order:")  
for i in range(n):  
    arr.append(int(input ()))  
  
key = int (input ("Enter the element to search: "))  
result = binary_search(arr, key)  
if result != -1:
```



```
print("Element found at index", result)
```

else:

```
print("Element not found")
```

OUTPUT:

```
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(b(i)).py =====
Enter number of elements: 5
Enter elements in sorted order:
11
22
33
44
55
Enter the element to search: 44
Element found at index 3
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(b(i)).py =====
Enter number of elements: 5
Enter elements in sorted order:
11
22
33
44
55
Enter the element to search: 66
Element not found
>>>
```

EXPLANATION:

- The program defines a `binary_search()` function that takes a sorted list and a search element as input.
- The variables `low` and `high` are initialized to represent the first and last positions of the list.
- The program calculates the middle position using `mid = (low + high) // 2` and compares the middle element with the search element.
- If the search element is greater or smaller than the middle element, the program searches only the required half of the list.
- If the element is found, its index is returned; otherwise, `-1` is returned and **"Element not found"** is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error if the statements inside the function or loop are not properly indented.	Maintain proper indentation inside the function and loop.
2.	ValueError if a non-numeric value is entered instead of an integer.	Enter only valid integer values.
3.	Incorrect result if the elements are not entered in sorted order.	Enter the elements in sorted order before performing binary search.
4.	IndexError if an invalid index is accessed while searching.	Make sure low and high remain within the list range.

B(ii). AIM: Write a python program to implement binary search without sorted(unsorted) array as an input

PROGRAM:

```
def binary_search(arr, key):

    arr.sort()

    low = 0

    high = len(arr) - 1

    while low <= high:

        mid = (low + high) // 2

        if arr[mid] == key:

            return mid

        elif arr[mid] < key:

            low = mid + 1

        else:

            high = mid - 1

    return -1

n = int (input ("Enter number of elements: "))

arr = []

print ("Enter elements:")

for i in range(n):

    arr.append(int(input()))

key = int (input ("Enter the element to search: "))

result = binary_search(arr, key)
```

```
if result != -1:

    print ("Element found at index", result)

else:

    print ("Element not found")
```

OUTPUT:

```
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(b(ii)).py =====
Enter number of elements: 5
Enter elements in unsorted order:
45
33
2
5
18
Enter the element to search: 33
Element found at index 3
>>>
===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab2(b(ii)).py =====
Enter number of elements: 5
Enter elements in unsorted order:
45
33
2
5
18
Enter the element to search: 55
Element not found
>>>
```

EXPLANATION:

1. The `binary_search()` function is defined to search for the given key in the list `arr`.
2. The `arr.sort()` function arranges the elements in ascending order so that binary search can be performed.
3. The variables `low` and `high` represent the first and last positions of the sorted list.
4. The program finds the middle element and compares it with the key. Based on the comparison, it searches either the left half or the right half.
5. If the element is found, its index is returned. If the element is not found, the function returns `-1`.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError may occur if while, if, or other statements are not properly aligned.	Maintain proper indentation inside the function and conditional blocks.
2.	SyntaxError may occur if – is used instead of the normal minus operator - in high = mid – 1.	Use the normal hyphen: high = mid - 1.
3.	ValueError occurs if a non-numeric value is entered instead of an integer.	Enter only valid integer values.
4.	NameError occurs if variables such as arr, key, or result are misspelled.	Check the spelling and define variables before using them.

LAB-3

A.AIM: Write a python program to implement bubble sort, where the input elements are taken from the user.

PROGRAM:

```
#bubble sort

def bubble_sort(arr):

    n = len(arr)

    for i in range (n-1):

        for j in range(n-i-1):

            if arr[j]>arr[j+1]:

                arr[j],arr[j+1] = arr[j+1],arr[j]

    return arr

#taking input

n = int(input("Enter number of elements: "))

arr=[]

print("Enter elements: ")

for i in range(n):

    arr.append(int(input()))

bubble_sort(arr)

print("sorted array: ")

for element in arr:

    print(element, end=" ")
```

OUTPUT:

```
>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab3(1) bubble sort.py =====
Enter number of elements: 5
Enter elements:
12
65
33
25
1
sorted array:
1 12 25 33 65
>>>
```

EXPLANATION:

- The `bubble_sort()` function is defined to arrange the elements of the list in ascending order.
- The outer for loop controls the number of passes, while the inner for loop compares adjacent elements.
- The condition `arr[j] > arr[j+1]` checks whether the current element is greater than the next element.
- If the elements are in the wrong order, they are exchanged using `arr[j], arr[j+1] = arr[j+1], arr[j]`.
- After all comparisons and swaps are completed, the sorted array is returned and displayed element by element.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values and linely
2.	NameError if <code>arr</code> , <code>n</code> , or another variable is misspelled.	Check the variable names and use the correct spelling.
3.	TypeError may occur if the list contains incompatible data types during comparison.	Enter elements of the same suitable data type, such as integers.
4.	IndexError may occur if the program accesses an invalid array position.	Use the correct loop range <code>range(n-i-1)</code> to keep the index within the list.

B.AIM: Write a python program to implement insertion sort, where the input elements are taken from the user.

PROGRAM:

#Insertion Sort

```
def insertion_sort(arr): n = len(arr) for i in range (1, n): key = arr[i] j = i - 1
```

```
    while j >= 0 and arr[j] > key:
```

```
        arr[j + 1] = arr[j]
```

```
        j = j - 1
```

```
    arr[j + 1] = key
```

```
return arr
```

#Taking input

```
n = int (input ("Enter number of elements: ")) arr = []
```

```
print ("Enter elements: ") for i in range(n): arr.append(int(input()))
```

#Sorting the array

```
insertion_sort(arr)
```

#Displaying sorted array

```
print ("Sorted array: ") for element in arr: print (element, end=" ")
```

OUTPUT:

```
==== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab3(2) insertion sort.py ====
Enter number of elements: 6
Enter elements:
42
14
35
54
33
29
Sorted array:
14 29 33 35 42 54
>>>
```


EXPLANATION:

- The `insertion_sort()` function is defined to arrange the elements of the list in **ascending order**.
- The for loop starts from the second element because the first element is considered already sorted.
- The variable `key` stores the current element, and `j` is used to compare it with the previous elements.
- The while loop shifts elements that are greater than `key` one position to the right.
- The `key` is placed in its correct position, and after all elements are processed, the sorted array is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the statements inside the function, for, or while loop are not properly indented.	Maintain proper indentation inside all blocks.
2.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values.
3.	NameError if variables such as <code>arr</code> , <code>key</code> , or <code>j</code> are misspelled.	Check the variable names and use the correct spelling.
4.	IndexError if an invalid list index is accessed while shifting elements.	Make sure <code>j >= 0</code> is checked before accessing <code>arr[j]</code> .

C.AIM: Write a python program to implement selection sort, where the input elements are taken from the user.

PROGRAM:

#Selection Sort

```
def selection_sort(arr): n = len(arr)

for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j
    arr[i], arr[min_index] = arr[min_index], arr[i]
return arr
```

#Taking input

```
n = int(input("Enter number of elements: ")) arr = [] print("Enter elements: ")
for i in range(n): arr.append(int(input()))
```

#Sorting the array

```
selection_sort(arr)
```

#Displaying sorted array

```
print("Sorted array: ")
```

```
for element in arr:
```

```
    print(element, end=" ")
```

OUTPUT:

```

>>> ===== RESTART: /Users/hasini/Documents/DSA CSE25133/Lab3(3) selection sort.py =====
Enter number of elements: 6
Enter elements:
22
42
12
2
72
32
Sorted array:
2 12 22 32 42 72
>>>

```

EXPLANATION:

- The `selection_sort()` function is defined to arrange the elements of the list in **ascending order**.
- The outer for loop selects the position where the smallest element should be placed.
- The variable `min_index` stores the index of the smallest element found in the unsorted part of the list.
- The inner for loop compares the elements and updates `min_index` whenever a smaller element is found.
- The smallest element is swapped with the element at position `i`, and after all passes, the sorted array is displayed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the function or loop statements are not properly indented.	Maintain proper indentation inside the function and loops.
2.	ValueError if a non-integer value is entered as an array element.	Enter only valid integer values.
3.	NameError if variables such as <code>arr</code> , <code>i</code> , <code>j</code> , or <code>min_index</code> are misspelled.	Check the variable names and use the correct spelling.
4.	IndexError if an invalid list index is accessed.	Use the correct loop ranges and ensure the index is within the list range.